

React JS Hands-on Lab

Unit Testing and Mocking with Jest – Git Client App

Objective

As an intern, the task is to create and test a React application which fetches and displays a list of repository names for a given GitHub user, named **gitclientapp**.

In this lab I have learnt how to:

- Unit test modules in isolation
- Creating and configuring mocks and spies

Step 1: Create the React Project and Install axios

Created a new React application named **gitclientapp** using create-react-app, opened it in VS Code, and installed the **axios** library to call the GitHub API:

```
npx create-react-app gitclientapp
cd gitclientapp
npm install axios
```

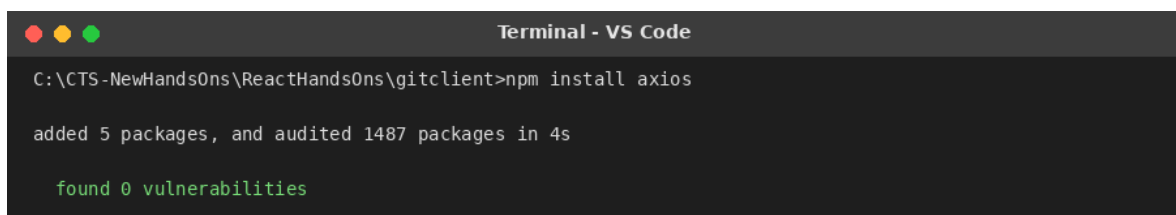


Figure 2: Install axios library

Step 2: GitClient Module

Created a new file **GitClient.js** in the src folder with a class **GitClient** that calls api.github.com to fetch the repositories of a given user:

```
import axios from "axios";

class GitClient {
  static getRepositories(userName) {
    const url = `https://api.github.com/users/${userName}/repos`;
    return axios.get(url);
  }
}

export default GitClient;
```

Figure 3: GitClient module (src/GitClient.js)

Step 3: App Component

Modified **App.js** to use the GitClient module to fetch and display the repositories of the user "techiesyed":

```
import './App.css';
import GitClient from './GitClient';
```

```
import { useEffect, useState } from 'react';

function App() {
  const [repositories, setRepositories] = useState([]);

  useEffect(() => {
    GitClient.getRepositories('techiesyed').then(r => setRepositories(r.data));
  }, [])

  return (
    <div className="App">
      <h1>Git repositories of User - TechieSyed</h1>
      {repositories.map(r =>
        <p key={r.name}>{r.name}</p>
      )}
    </div>
  );
}

export default App;
```

Figure 4: App Component (src/App.js)

Step 4: Build and Run the Application

Built and ran the application using **npm start**. The browser at localhost:3000 shows the list of repository names fetched from the GitHub API for the user techiesyed:

npm start

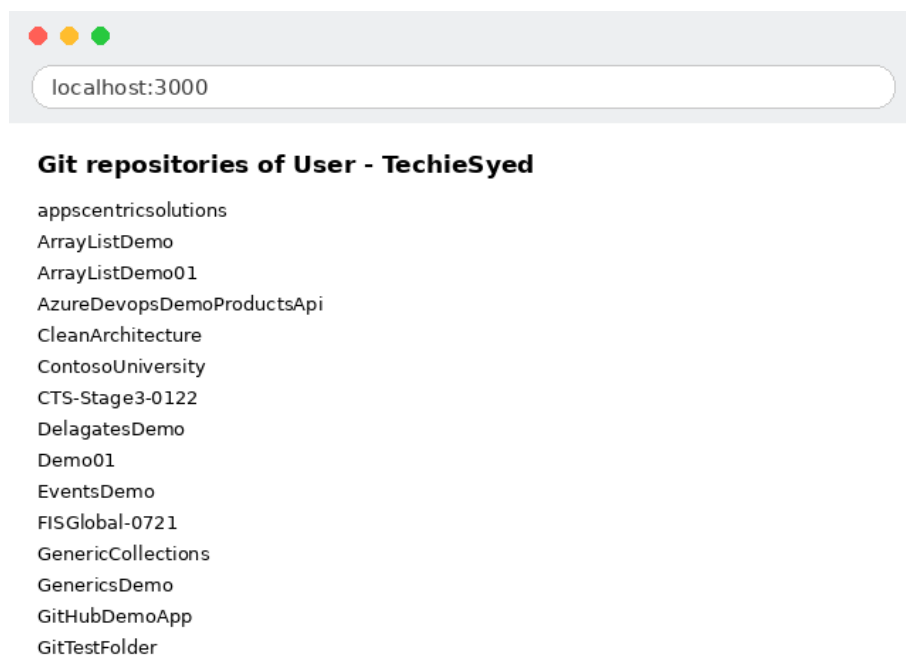


Figure 5: Application Output

Step 5: Unit Test with Jest and Mocking axios

Created **GitClient.test.js**, imported axios and GitClient, mocked axios using **jest.mock('axios')** so the test does not make a real network call, and verified that **getRepositories()** returns the mocked data:

```
import axios from 'axios';
```

```

import GitClient from './GitClient';

jest.mock('axios');

describe('Git Client Tests', () => {
  test('should return repository names for techiesyed', () => {
    const repos = {
      data: [
        { name: 'repo1' },
        { name: 'repo2' }
      ]
    };
    axios.get.mockResolvedValue(repos);

    return GitClient.getRepositories('techiesyed').then(response => {
      expect(response.data.length).toBe(2);
      expect(response.data[0].name).toBe('repo1');
      expect(response.data[1].name).toBe('repo2');
    });
  });
});

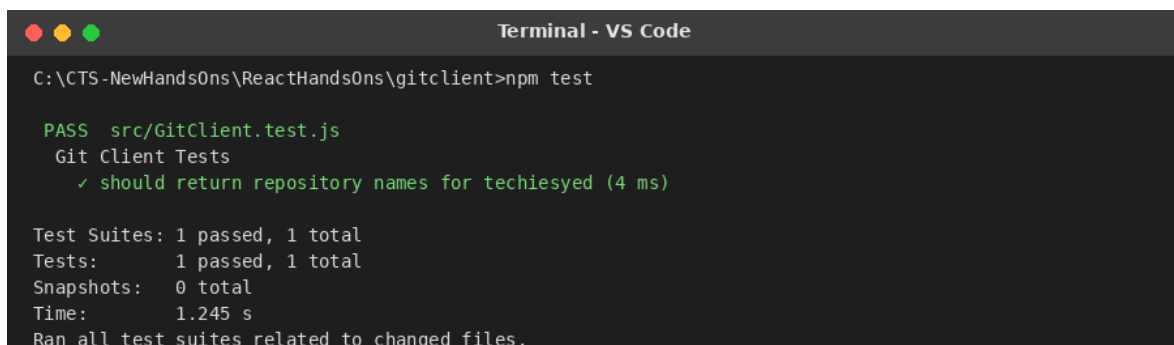
```

Code: src/GitClient.test.js

Step 6: Run the Tests

Ran the unit tests using the following command:

```
npm test
```



The image shows a terminal window titled "Terminal - VS Code" with the following output:

```

C:\CTS-NewHands0ns\ReactHands0ns\gitclient>npm test

PASS src/GitClient.test.js
  Git Client Tests
    ✓ should return repository names for techiesyed (4 ms)

Test Suites: 1 passed, 1 total
Tests:      1 passed, 1 total
Snapshots:  0 total
Time:       1.245 s
Ran all test suites related to changed files.

```

Figure 6: Run tests

End of Lab